Round 6.1: WSL Local Reproduction and Go/No-Go Review

UFUG2106 Truth Table to Circuit — Controlled Cut Rewriting

Local experiment analysis

11 November 2026

Executive summary

Research decision: NO-GO for promoting Round 6.1 to the default synthesizer. Retain the verified Round 5.1 baseline and archive Round 6.1 as a research branch. Round 6.1 passed all official and fresh holdout functionality and timing checks, and reduced Area consistently, but **failed the pre-registered synthesis-runtime criterion** of at most 1.30 times the Round 5.1 median runtime on the paired machine.

The source of evidence is the uploaded WSL archive `20261111_184840_Round6.1_local_results.zip`, created on 11 November 2026. All numbers in this report are calculated from the archived evaluator CSVs, validation logs, and the experiment manifest. No additional synthesis run was performed for this report.

Main finding	Result
Official Stage 1 evaluator calls	100/100 PASS and 100/100 target met
Two-run reference Area	529,014 / 529,014
Two-run Round 6.1 Area	528,854 / 528,839
Mean Area reduction relative to reference	167.5 (0.0317%)
Mean of per-run median runtime: reference	52.398 ms
Mean of per-run median runtime: Round 6.1	77.407 ms
Median runtime increase	+47.73% (limit: +30%)
New holdout evaluator calls	96/96 PASS and target met
Independent Round 6.1 netlist verification	73/73 PASS; 55 exhaustive, 18 sampled

1. Experimental design and controls

The WSL environment reported Linux `6.6.87.2-microsoft-standard-WSL2`, GCC `13.3.0`, and Python `3.12.3`. Both implementations were compiled on the same system and tested on the same 25 released Stage 1 truth tables. There were **two paired runs with reversed execution order**, reducing, but not eliminating, potential systematic machine-load bias. The archive includes source snapshots, SHA-256 hashes of test cases and source files, official CSV and evaluator logs, and saved first-run netlists.

The reference was **Round 5.1**. The Round 6.1 modification is a postprocessing cut rewriter with at most four leaves per cut. The design and time budget were fixed before the 48 new holdout cases were evaluated. The stated decision criteria were:

1. All 25 released cases must PASS and meet the specified circuit delay.
2. Area must improve strictly over Round 5.1.
3. Newly frozen holdouts must remain functionally correct.

4. Paired median synthesis runtime must not exceed **1.30 times** Round 5.1.

A *functional PASS* and a *target delay met* are distinct official fields. Runtime here is the evaluator-recorded **synthesizer execution time**, not the delay of the output circuit.

2. Stage 1: complete paired evaluation

Metric	Run 1 R5.1	Run 1 R6.1	Run 2 R5.1	Run 2 R6.1
PASS	25/25	25/25	25/25	25/25
Target met	25/25	25/25	25/25	25/25
Area sum	529,014	**528,854**	529,014	**528,839**
Median runtime (ms)	54.572	80.731	50.224	74.083
Runtime sum (ms)	7,371.348	6,154.935	5,182.265	5,875.605
Area-improved cases	—	10	—	9
Area-regressed cases	—	0	—	0
Cases with lower runtime	—	9	—	1

The Area reduction was **160** units in Run 1 and **175** units in Run 2. The median runtime ratios were 1.477 on the pooled two-run mean-of-medians basis, 1.479 in Run 1 and 1.475 in Run 2. All three comparisons exceed the predetermined 1.30 threshold.

Why the total runtime gives a mixed signal. In Run 1 the full 25-case runtime sum fell by approximately 16.5%, partly because a few very large instances completed substantially faster. In Run 2 the total increased by 13.4%. Since only one case became faster in Run 2, and the median increased in both runs, the evidence does **not** support a general runtime improvement.

2.1 Where did the Area savings occur?

These are the eight largest reductions in Run 1:

Case	R5.1 Area	R6.1 Area	Saved	Delay (R5.1 → R6.1)
case_010	4,712	4,661	51	32 → 32
case_012	5,359	5,309	50	25 → 25
case_019	5,319	5,299	20	57 → 57
case_003	24,251	24,235	16	56 → 56
case_014	11,662	11,650	12	61 → 61
case_013	11,252	11,248	4	60 → 60
case_001	61,952	61,950	2	62 → 62
case_005	700	698	2	31 → 31

The only Area changes between the two Round 6.1 replicates were `case_005` (+2 units on Run 2 relative to Run 1), `case_010` (+3), and `case_012` (−20), resulting in **15 fewer Area units** overall in Run 2. Both runs remained functionally correct and within target. This is consistent with wall-clock-bounded candidate search; it is not by itself evidence of a Boolean correctness failure.

3. Holdout evaluation and generalization

The archive contains the frozen `holdout_round61_fresh_v1` (40 cases) and `holdout_round61_large_v1` (8 cases), which were not used to select the 25 ms cut budget or optimize the heuristic. They are synthetic functions, not an empirical sample of hidden Stage 2. Both programs PASSed and met the target delay on every case.

Holdout	R5.1 Area	R6.1 Area	Area reduction	R5.1 median (ms)	R6.1 median (ms)
Fresh mixed (40)	11,186	**10,858**	328 (2.93%)	7.852	10.762
Fresh larger (8)	67,275	**67,047**	228 (0.34%)	68.093	94.327

Area improved on **16/40** mixed and **5/8** large cases; no case regressed in Area. The median runtime increased by approximately **37.1%** and **38.5%**, respectively. Thus, the independent holdouts strengthen the evidence for **generalizable Area-reduction opportunities**, but do not establish a favorable Area–Runtime tradeoff.

An additional caveat: repeated evaluation of the same holdout would turn it into development data. Preserve these holdouts as an evaluation set; do not tune a Round 6.2 threshold against them while still calling them blind.

4. Mathematical method and correctness controls

4.1 Four-feasible cuts

For a DAG node v, a *cut* C intersects every path from v backwards to the primary inputs. Round 6.1 uses $|C| \leq 4$. Treating the cut leaves as formal Boolean variables, we evaluate the original cone and obtain its complete small truth mask:

$$M_{v,C} = \sum_{s=0}^{2^{|C|}-1} f_{v,C}(s)\, 2^s.$$

Because the cut contains at most four formal inputs, the mask needs at most 16 bits. This remains valid even if actual leaf signals are logically dependent: equivalence is verified for all **formal** assignments, which is stronger than equivalence restricted to reachable assignments.

4.2 Cache and candidate search

The symbolic greedy prime-implicant cover is cached under the tuple (k, M, p), with $p \in \{0, 1\}$ denoting one of two deterministic cover policies. The cache contains symbolic cubes only—not concrete gate IDs or timing information. Any cached result is instantiated against the new cut leaves, avoiding a dependency on the gate numbering of a previous candidate.

Candidate forms include SOP, complemented SOP/POS, literal factoring and fixed-order Shannon decompositions. Cuts are prioritized by conservatively estimated *private reclaimable area*. Unlike the earlier Round 6.0 prototype, Round 6.1 places a fixed 25 ms time budget (also constrained by a fraction of the synthesizer time limit) on the cut search, rather than increasing the search space. The precise source documents clarify that the cut-budget cap does not bound all synthesis or graph-serialization work.

4.3 Whole-circuit safety acceptance

The postprocessor may propose a locally equivalent subgraph, but the accepted output must pass a strict full-circuit check:

$$A(G_{new}) < A(G_{incumbent}), \qquad D(G_{new}) \leq D_{target}.$$

This avoids confusing local gate count with true DAG area when intermediate nodes have multiple consumers. The full algorithm also retains a valid incumbent candidate if its bounded cut search finds nothing better. It cannot guarantee a lower Area against a **different** run of the same timed baseline, since candidate search can be affected by machine scheduling.

A standard structural-induction argument proves equivalence of cut truth functions and their SOP, POS, factoring and Shannon forms; replacing a gate only by an equivalent expression in its cut boundary preserves the external outputs. Official evaluator results and an independent verifier provide empirical checks of the implementation, not a substitute for a formal whole-program proof.

5. Independent validation and provenance

The independent log contains **73/73** Round 6.1 successful netlist verifications: the 25 official Stage 1 outputs plus 48 new holdout outputs. It records **55 exhaustive** checks and **18 deterministic sampled** checks. Sampled verification is not an exhaustive correctness proof for larger functions.

The original local experiment ZIP contains, among other artifacts:

- Four official evaluator CSVs and logs, first-run netlists, per-case comparison, and independent checker log.
- Four holdout evaluator CSVs and logs plus their netlists.
- Source snapshot, build log, compiler/OS information, evaluator and source SHA-256 hashes, and test-manifest hashes.
- Original mathematical design and container-reference experiment notes. Container and WSL runtime measurements must not be conflated.

The accompanying `Round6.1_Stage1_TwoRun_Comparison.csv`, `Round6.1_Holdout_Comparison.csv` and `Round6.1_Local_Statistics.json` were derived from the uploaded raw CSV files without changing the original result archive.

6. Decision: stop this specific optimization branch

Pre-registered decision: NO-GO for default promotion. The official correctness and target-delay constraints hold; new holdouts show additional Area reduction, and Area never regressed in the measured examples. However, the **1.30× median-runtime requirement was missed**: observed mean-of-medians ratio was **1.477×**. The average Area gain was only **0.0317%** on Stage 1.

We should:

1. **Freeze Round 5.1 as the default efficiency baseline**, rather than replacing it with Round 6.1.
2. **Archive Round 6.0 and 6.1 as valid research explorations** demonstrating area gains but unfavorable Stage 1 runtime tradeoffs.
3. **Avoid a Round 6.2 cut-budget sweep** unless the course's scoring rule clearly rewards an approximately 0.03% Area gain enough to justify a roughly 48% median-runtime penalty, or a fundamentally different implementation removes the dominant cost.
4. If further work becomes justified, investigate more efficient *in-memory candidate construction*, sharing-aware delta-cost estimation, or a precomputed tiny-function library—but treat these as a new hypothesis with fresh frozen tests, not iterative tuning on the current blind holdouts.

The actual final submission should be selected only after reading the official scoring formula and its synthesizer runtime rules. No course scoring formula was verified as part of this analysis.